

FeCUDA — Reporte de Bugs, Validación y Profiling

Proyecto: FeCUDA / forgethreads **GPU objetivo:** NVIDIA GTX 1650 (Turing, sm_75) **Fecha:** 2026-03-17

1. Diagnóstico inicial

El usuario reportó que la librería retornaba un número variable de paths entre ejecuciones: aproximadamente ± 5 resultados distintos con los mismos datos de entrada. Se realizó un análisis completo del código fuente antes de hacer ningún cambio.

Estructura del proyecto revisada

src/	
main.cu	← test harness
bindings.cu	← módulo Python (pybind11)
algorithms/maxmin.cu	← lógica de lanzamiento del kernel
kernels/maxmin/kernel_v1.cu	← kernel CUDA principal
kernels/algorithms/maxmin.cu	← ARCHIVO DUPLICADO (bug de build)
include/	
core/types.cuh	← TensorResult<T>
kernels/maxmin_kernels.cuh	
CMakeLists.txt	

2. Bugs encontrados y corregidos

Se identificaron **6 bugs** en total: 3 críticos, 1 de build y 2 menores.

Bug 1 — Race condition en el kernel (causa del ± 5 paths)

Archivo: `src/kernels/maxmin/kernel_v1.cu` **Severidad:** Crítico — produce resultados no deterministas

Descripción

El kernel `maxmin_threshold_kernel` se lanza con `dim3 grid(N, M, B)`, lo que significa que todos los bloques de la misma fila `m` corren en paralelo. El problema es que:

1. El bloque `(b, m, n)` lee `X[b, m, k]` para `k = 0..K-1` (toda la fila `m`)
2. Al final, el bloque **escribe** `X[b, m, n] = k_max` (escribe a la misma fila `m`)

Entonces el bloque `(b, m, n0)` puede leer `X[b, m, n1]` **después** de que el bloque `(b, m, n1)` ya sobrescribió ese valor con su `k_max`. Esto cambia el resultado del `min(X[a_idx], X0[b_idx])`, lo que altera cuál es el `k_max` calculado y, en consecuencia, cuántos paths superan el umbral.

```
Bloque (m=3, n=5): lee X[b,3,k] para k=0..K-1
Bloque (m=3, n=7): lee X[b,3,k] para k=0..K-1
Bloque (m=3, n=5): escribe X[b,3,5] = k_max_5 ← sobrescribe
Bloque (m=3, n=7): ahora lee X[b,3,5] = k_max_5 ← ve dato modificado!
```

Fix aplicado

Cambiar todas las lecturas del tensor mutable `X` por lecturas del tensor constante `X0` (copia original que nunca se modifica). Solo se conserva la escritura final a `X`:

```
// ANTES (con race condition):
v = __hmax(v, __hmin(X[a_idx], X0[b_idx]));
if (__hsub(k_max, X[out_id]) >= thr)
    __half mi = __hmin(X[a_idx], X0[b_idx]);

// DESPUÉS (correcto – todas las lecturas desde X0):
v = __hmax(v, __hmin(X0[a_idx], X0[b_idx]));
if (__hsub(k_max, X0[out_id]) >= thr)
    __half mi = __hmin(X0[a_idx], X0[b_idx]);

// La única escritura a X se mantiene:
if (tid == 0) X[out_id] = k_max;
```

Bug 2 — Doble offset en el camino BATCHED

Archivo: `src/algorithms/maxmin.cu` **Severidad:** Crítico — produce resultados incorrectos (mitad de paths) y accesos out-of-bounds

Descripción

Cuando $B \times M \times N \times K > \text{MAX_PATHS_PER_ITER } (100,000)$, el algoritmo usa un camino "batched" que lanza el kernel una vez por batch:

```
__half *localA = d_A + (b_ * sizeA); // puntero YA desplazado al batch b_
__half *localB = d_B + (b_ * sizeB);
maxmin_threshold_kernel<<<grid, block, shmem>>>(
    localA, localB, ..., b_           // batch_id = b_ ← BUG
);
```

Dentro del kernel, el índice se calcula como $a_idx = b * M * K + m * K + k$ donde $b = \text{batch_id} = b_$. El acceso real es:

```
localA[b_ * M * K + m * K + k]
= (d_A + b_ * sizeA)[b_ * M * K + m * K + k]
= d_A[2 * b_ * M * K + m * K + k] ← DOUBLE OFFSET
```

Consecuencia observada: B=8, M=32 → CPU=4808 paths, GPU=2404 (exactamente la mitad). Para batches con índice alto, el acceso quedaba fuera de los límites del buffer.

Fix aplicado

Pasar `batch_id=0` y `B=1` al kernel cuando se usa el camino batched:

```
maxmin_threshold_kernel<<<grid, block, shmem>>>(
    localA, localB, ...,
    1, // B=1: el kernel ve solo 1 batch
    M, N, K,
    0 // batch_id=0: el puntero ya apunta al batch correcto
);
```

Bug 3 — División por cero en TensorResult con dims=0

Archivo: `include/core/types.cuh` **Severidad:** Crítico — SIGFPE
crash cuando count=0 (ej: thr muy alto sin paths)

Descripción

`TensorResult::allocateData()` realiza una comprobación de overflow con una división:

```
if (dest > std::numeric_limits<int>::max() / mult)
```

Cuando se crea `TensorResult(Device, 0, 4, 1, 1)` (count=0 paths), con `ds = {0, 4, 1, 1}`: - En la iteración `ix=1`:

`ds_copy = {0, 1, 1}`, `mult = 0 * 1 * 1 = 0` - La expresión `INT_MAX / 0` produce **SIGFPE** (división por cero en C)

Esto se disparaba en Python cuando el threshold era lo suficientemente alto para no encontrar paths.

Fix aplicado

```
// Si mult==0 el producto total es 0 (válido), no hay riesgo de overflow
if (mult == 0) continue;
if (dest > std::numeric_limits<int>::max() / mult)
```

Bug 4 — Archivo duplicado mal ubicado (falla de compilación)

Archivo: `src/kernels/algorithms/maxmin.cu` **Severidad:** Build failure

Era una copia de `src/algorithms/maxmin.cu` en la ruta incorrecta. Sus includes relativos apuntaban a `src/include/` que no existe. El `CMakeLists.txt` lo incluía via `GLOB_RECURSE`, causando un error fatal de compilación.

Fix: Archivo eliminado.

Bug 5 — CMakeLists.txt: doble BUILD_TYPE y flag -G

Archivo: `CMakeLists.txt` **Severidad:** Menor (impacto en rendimiento y confusión)

1. `CMAKE_BUILD_TYPE` declarado dos veces: `Debug` en línea 11 y `RelWithDebInfo` en línea 45.
2. Flag `-G` (debug completo de device code) activo, lo que **desactiva todas las optimizaciones CUDA**.

Fix: Eliminado el `set(CMAKE_BUILD_TYPE Debug)` redundante. Reemplazado `-G` por `-lineinfo` (conserva números de línea para profilers sin deshabilitar optimizaciones).

Bug 6 — `globalId` incorrecto en `FloatToHalfKernel`

Archivo: `src/main.cu` **Severidad:** Menor (solo afecta al test harness)

```
// ANTES (incorrecto):
int globalId = blockIdx.x * gridDim.x + threadIdx.x;

// DESPUÉS (correcto):
int globalId = blockIdx.x * blockDim.x + threadIdx.x;
```

3. Validación — compute-sanitizer

Se ejecutaron los 4 modos del sanitizer contra el binario compilado en modo RelWithDebInfo con `-lineinfo`:

```
compute-sanitizer --tool memcheck --leak-check=full ./fecuda_main
compute-sanitizer --tool racecheck ./fecuda_main
compute-sanitizer --tool initcheck ./fecuda_main
compute-sanitizer --tool synccheck ./fecuda_main
```

Herramienta	Resultado
memcheck	0 errors, 0 bytes leaked
racecheck	0 hazards (0 errors, 0 warnings)
initcheck	0 errors
synccheck	0 errors

Sin errores en ningún modo.

4. Validación — CPU reference vs GPU

Se implementó una referencia CPU exacta del algoritmo MaxMin y se comparó contra la GPU en 13 configuraciones × 3 runs por configuración.

Referencia CPU

```
for b, m, n:
    k_max = max_k min(A[b,m,k], A[b,k,n])
    if (k_max - A[b,m,n]) >= thr:
        count += paths k where |min(A[b,m,k],A[b,k,n]) - k_max| <= 0.01
```

Resultados (C++ / fecuda_main)

B	M	thr	Camino	CPU	GPU	Estado
1	4	0.30	COMPLETE	0	0	✓ OK
1	8	0.30	COMPLETE	26	26	✓ OK

B	M	thr	Camino	CPU	GPU	Estado
1	16	0.30	COMPLETE	131	131	✓ OK
1	32	0.30	COMPLETE	601	601	✓ OK
1	64	0.30	BATCHED	2723	2723	✓ OK
1	128	0.30	BATCHED	12218	12218	✓ OK
1	16	0.00	COMPLETE	232	232	✓ OK
1	16	0.50	COMPLETE	80	80	✓ OK
1	16	0.90	COMPLETE	1	1	✓ OK
4	16	0.30	COMPLETE	524	524	✓ OK
8	32	0.30	BATCHED	4808	4808	✓ OK
10	16	0.40	COMPLETE	1060	1060	✓ OK
30	64	0.30	BATCHED	81690	81690	✓ OK

Resultado: TODOS OK — 13 configs × 3 runs = 39 ejecuciones sin variación.

5. Validación — tests Python con NumPy

El módulo forgethreads fue testeado desde Python con tensores NumPy (float16) usando el protocolo DLPack nativo de NumPy ≥ 1.22.

Test 1: Correctitud GPU vs CPU reference

B	M	thr	CPU	GPU	Estado
1	4	0.30	5	5	✓ OK
1	8	0.30	23	23	✓ OK
1	16	0.30	133	133	✓ OK
1	32	0.30	623	623	✓ OK
1	16	0.00	226	226	✓ OK
1	16	0.50	77	77	✓ OK
1	16	0.90	0	0	✓ OK
4	16	0.30	510	510	✓ OK
8	16	0.30	1010	1010	✓ OK

Test 2: Determinismo (5 runs por configuración)

Config	Runs	Estado
B=1 M=16 thr=0.3	[133, 133, 133, 133, 133]	✓ OK

Config	Runs	Estado
B=1 M=64 thr=0.3	[2744, 2744, 2744, 2744, 2744]	✓ OK
B=4 M=32 thr=0.3	[2427, 2427, 2427, 2427, 2427]	✓ OK
B=8 M=32 thr=0.3	[4893, 4893, 4893, 4893, 4893]	✓ OK

Test 3: Exploración de dimensiones

Variando M (B=1):

M	Camino	Tiempo	Estado
1	COMPLETE	5.5 ms	✓ OK
2	COMPLETE	3.8 ms	✓ OK
4	COMPLETE	4.4 ms	✓ OK
8	COMPLETE	5.4 ms	✓ OK
16	COMPLETE	5.1 ms	✓ OK
32	COMPLETE	4.2 ms	✓ OK
64	BATCHED	5.1 ms	✓ OK
128	BATCHED	5.8 ms	✓ OK
256	BATCHED	10.9 ms	✓ OK

Variando B (M=16):

B	Camino	Tiempo	Estado
1	COMPLETE	3.4 ms	✓ OK
2	COMPLETE	5.1 ms	✓ OK
4	COMPLETE	5.4 ms	✓ OK
8	COMPLETE	5.0 ms	✓ OK
16	COMPLETE	3.6 ms	✓ OK
32	BATCHED	5.5 ms	✓ OK
64	BATCHED	6.9 ms	✓ OK
128	BATCHED	11.4 ms	✓ OK

Resultado: 36/36 checks OK.

6. Límites de entrada de la librería

Restricciones de la operación

Parámetro	Restricción	Motivo
Forma tensor	[B, M, N] con M == N	

Parámetro	Restricción	Motivo
		El kernel indexa ambas matrices como <code>[B, N, N]</code> . Si $M \neq N$ los índices son incorrectos.
<code>ndim</code>	<code>>= 3</code>	El constructor <code>TensorResult(py::object)</code> requiere al menos 3 dimensiones.
Tipo de datos	<code>float16 / bfloat16</code>	El kernel usa <code>__half</code> nativo de CUDA.
Threshold <code>thr</code>	<code>[0.0, 1.0]</code> para datos en <code>[0, 1]</code>	No hay límite estricto, pero valores fuera de rango producen 0 o todos los paths.

Límite: camino COMPLETE vs BATCHED

El umbral `MAX_PATHS_PER_ITER = 100,000` en `algorithms/maxmin.cu` determina qué código se ejecuta:

- **COMPLETE** (lanzamiento único): cuando $B \times M \times N \times K < 100,000$
- Equivalente con matrices cuadradas: $B \times M^3 < 100,000$
- Ejemplo: $B=1, M \leq 46$ | $B=10, M \leq 21$
- **BATCHED** (lanzamiento por batch): cuando $B \times M^3 \geq 100,000$
- Se lanza el kernel B veces, una por batch

Límite de memoria GPU (GTX 1650, 4 GB VRAM)

M	VRAM input (2 tensors, float16)	VRAM output máx (BATCHED)	Factible
64	$2 \times 64^2 \times 2 B = 16 \text{ KB}$	$64^3 \times 18 B = 4.5 \text{ MB}$	✓
128	$2 \times 128^2 \times 2 B = 65 \text{ KB}$	$128^3 \times 18 B = 36 \text{ MB}$	✓
256	$2 \times 256^2 \times 2 B = 262 \text{ KB}$	$256^3 \times 18 B = 288 \text{ MB}$	✓
512	$2 \times 512^2 \times 2 B = 1 \text{ MB}$	$512^3 \times 18 B = 2.3 \text{ GB}$	⚠ límite
1024	$2 \times 1024^2 \times 2 B = 4 \text{ MB}$	$1024^3 \times 18 B = 18 \text{ GB}$	✗ OOM

M máximo práctico ≈ 450 para una sola batch con 4 GB de VRAM.

Límites de grilla CUDA (hardware)

Para el camino COMPLETE, la grilla es `dim3(N, M, B)` : - $N, M \rightarrow \text{máx } 65,535$ (pero la memoria limita antes) - $B \rightarrow \text{máx } 65,535$ (grid.z en `sm_75`)

Para el camino BATCHED, la grilla es `dim3(N, M, 1)`, sin límite de batch.

7. Análisis de ocupancia del kernel

A partir de la compilación con `--ptxas-options=-v`:

Métrica	Valor
Registros por thread	22
Shared memory por bloque	256 bytes (128 × sizeof(__half))
Tamaño de bloque	128 threads (4 warps)
Warps limitantes (por registros)	32 / 32
Warps limitantes (por smem)	32 / 32
Ocupancia teórica	100%

Con 22 registros/thread y bloque de 128 threads: - `65536 regs / (22 × 128) = 23 bloques/SM` → 92 warps → capped a 32 warps/SM ✓ - `65536 bytes smem / 256 bytes = 256 bloques/SM` → amplio margen ✓

La ocupancia teórica es 100% — configuración óptima para sm_75.

8. Análisis con Nsight Systems (nsys)

Se ejecutó `nsys profile --trace=cuda` sobre el binario completo de validación.

Resumen de CUDA API calls

API call	% tiempo	Nº llamadas	Avg (µs)	Observación
<code>cudaMalloc</code>	82.0%	461	389	Cuello de botella principal
<code>cudaDeviceSynchronize</code>	7.1%	173	89	Necesario tras cada kernel
<code>cudaFree</code>	5.7%	461	27	Simétrico a malloc
<code>cudaMemcpy</code>	2.6%	478	12	Transferencias pequeñas
<code>cudaLaunchKernel</code>	2.3%	173	29	Overhead de lanzamiento mínimo

Observación crítica: `cudaMalloc` consume el 82% del tiempo total de API. Esto se debe a que el código asigna y libera buffers de paths

en cada llamada. En un escenario de producción, se recomienda pre-allocar los buffers y reutilizarlos.

Resumen de kernels GPU

Kernel	% tiempo GPU	Instancias	Avg (µs)	Min (µs)	Max (µs)
maxmin_threshold_kernel	83.9%	147	84.7	2.5	599.8
randomInit	15.8%	13	180.3	34.7	761.9
randomFill	0.3%	13	3.3	1.7	10.4

El kernel principal ocupa el 83.9% del tiempo GPU, lo que es correcto. La variación Max/Min (2.5 µs vs 599.8 µs) refleja la diferencia entre configuraciones pequeñas (M=4) y grandes (M=128).

Nota sobre ncu (Nsight Compute)

El entorno de ejecución (contenedor Docker) tiene `RmProfilingAdminOnly=1` en el módulo del driver NVIDIA, lo que impide el acceso a hardware performance counters. Este parámetro solo puede modificarse en el sistema host con:

```
sudo modprobe nvidia NVreg_RestrictProfilingToAdminUsers=0
```

Para obtener métricas completas de coalescing, ocupancia real y throughput de DRAM, se recomienda ejecutar ncu en el sistema host con los permisos adecuados.

9. Benchmark GPU vs CPU

Config	CPU (ms)	GPU (ms)	Speedup	Paths
B=1 M=16	2.65	6.11	0.4×	133
B=1 M=32	14.28	4.98	2.9×	623
B=1 M=64	94.29	4.07	23×	2744
B=1 M=128	752.14	4.57	165×	12348
B=4 M=32	51.93	3.54	15×	2427
B=8 M=32	104.43	4.25	25×	4893

Para M pequeño (≤ 16), el overhead de transferencia CPU→GPU domina. A partir de M=32, la GPU supera a la CPU, con speedups que crecen cúbicamente ($O(M^3)$ en CPU vs $O(M^2)$ paralelo en GPU).

10. Resumen de cambios por archivo

Archivo	Cambio
<code>src/kernels/maxmin/kernel_v1.cu</code>	Fix Bug 1: lecturas <code>X[a_idx]</code> y <code>X[out_id]</code> → <code>X0[...]</code>
<code>src/algorithms/maxmin.cu</code>	Fix Bug 2: <code>batch_id=b_</code> → <code>batch_id=0, B=1</code> en camino batched
<code>include/core/types.cuh</code>	Fix Bug 3: <code>if (mult == 0) continue</code> antes de <code>INT_MAX / mult</code>
<code>src/kernels/algorithms/maxmin.cu</code>	Fix Bug 4: archivo eliminado (duplicado mal ubicado)
<code>CMakeLists.txt</code>	Fix Bug 5: eliminado <code>CMAKE_BUILD_TYPE Debug</code> redundante; <code>-G</code> → <code>-lineinfo</code>
<code>src/main.cu</code>	Fix Bug 6: <code>blockIdx.x * gridDim.x</code> → <code>blockIdx.x * blockDim.x</code> ; añadida validación CPU reference con 13 configs × 3 runs
<code>tests/test_numpy.py</code>	Nuevo: suite de tests Python con NumPy (36 checks)

11. Estado final

- **Determinismo:** resultados idénticos entre ejecuciones con misma entrada ✓
- **Correctitud:** 100% coincidencia con referencia CPU (C++ y Python) ✓
- **Sanitizers:** 0 errores en memcheck, racecheck, initcheck, synccheck ✓
- **Build:** compila sin errores ni warnings ✓
- **Rendimiento:** optimizaciones CUDA habilitadas (flag `-G` eliminado) ✓
- **Ocupancia teórica:** 100% en sm_75 ✓